

Case Study

Rover communication system

Contents

1. Introduction	4
2. How do you identify where to send to?	4
2.1. Why CAN has no addresses	4
2.2. First idea: sender and receiver in the ID	4
2.3. Problem	5
2.4. Solution	5
3. How do you define the structure of the message?	5
3.1. How to pack different data (just examples)	5
3.1.1. For example, Motor Drive Command (Command ID: 0x12)	5
3.1.2. Another example, receiving IMU Sensor Readings (Command ID: 0x35)	6
4. How do you handle receiving and processing packets at high speed?	6
4.1. Step 1: Let the hardware filter first	6
4.2. Step 2: Keep the interrupt as short as possible	6
4.3. Step 3: Ring buffer (circular buffer)	6
4.4. Step 4: Dispatch table instead of a long if/else chain	7
4.5. Step 5: Handle priority in software too	7
4.6. Step 6: Latest value vs every value	7
4.7. Step 7: Check every frame on receive	8
4.8. Summary of the full receive path	8
5. How do you handle multiple types of packets to send or receive?	8
5.1. Three kinds of messages	8
5.2. One list of all messages	9
5.2.1. For example	9
5.3. How regular messages are sent	9
5.4. How alert messages are sent	9
5.5. How question and answer messages are sent	9
5.6. Waiting in line to be sent	9
5.7. Receiving different messages	10
5.8. Don't overload the bus	10
6. How do you handle a change in requirements?	10
6.1. Leave empty space from the start	10
6.2. Change the message list first	10
6.2.1. For example, adding a temperature sensor	10
6.3. Add new things, don't change old things	11
6.4. Every board says its version	11
6.5. Check the bus before making things faster	11
6.6. Test the change before putting it on the rover	11
6.7. Make updating easy	11
7. Are there other things to keep in mind?	11
7.1. Wiring	12
7.2. Motor noise	12
7.3. Vibration	12
7.4. Broken boards	12
7.5. Safety	12
7.6. Timestamps	12

7.7. Bus health	12
7.8. Documentation	12
8. Scenario: the communication system is too slow	12
8.1. Part 1: Start the debugging	13
8.1.1. Step 1: Look at the bus	13
8.1.2. Step 2: Do the math	13
8.1.3. Step 3: Find where the messages are lost	13
8.1.4. Step 4: Check the wiring	13
8.1.5. Step 5: Test one board at a time	14
8.2. Part 2: If the problem persists	14
8.2.1. Send less	14
8.2.2. Do the work closer to the data	14
8.2.3. Get more room	14
8.2.4. Talk with the teams	14

1. Introduction

This case study is for the Robo-team interview with Ethan Bastian.

The assignment asked us to design a **general communication system** that allows all boards to communicate reliably and efficiently.

For starters, most vehicles nowadays use the CAN bus protocol for their internal communication, which we will dive into later in this document.

I picked CAN for this rover for four reasons:

- It handles electrical noise well, because it uses a differential pair instead of a single signal wire.
- It decides priority on its own: when two boards talk at the same time, the more important message wins automatically.
- It has built-in error checking (CRC) and automatic retransmission in hardware, so the CPU doesn't have to do it.
- It is the standard in vehicles, so the parts, tools and analyzers are cheap and easy to get.

In this document, I first go through the main points to think about, and at the end I describe how I would debug the scenario that was given.

2. How do you identify where to send to?

2.1. Why CAN has no addresses

CAN bus doesn't have a CS (Chip Select) pin, or an address you can call like in I2C. Instead, every message is broadcast to the whole bus, and each board picks up the messages whose ID tag it needs.

The idea works like a BLE service_id. In BLE you have a device_id and a service_id, but in CAN there is only one bus that every peripheral connects to, and each peripheral takes the data with the matching ID.

One detail that matters for the rest of this document: on CAN, the lowest ID wins the bus. When two boards start sending at the same time, the one with the numerically smaller ID keeps sending and the other one backs off and retries. That is exactly why I put the priority in the first bits of the ID: a lower priority number means a lower ID, which means it goes first.

2.2. First idea: sender and receiver in the ID

To make the CAN bus know where to send to, my first idea is to use the ID tag as a sender and receiver tag. I split the 11-bit ID like this:

- 3 first bits for the message type / priority.
- 4 next bits to identify the destination.
- 4 next bits to identify who sent it.

It looks like it fits perfectly, because both the sender and the receiver can be identified.

2.3. Problem

On paper it seems like a very nice idea. But what if there are more than 16 devices hooked onto one bus? The split above only allows 16 unique devices, because of the 4-bit identification.

You also miss the sweet spot of CAN, which is the shared data bus, because now there is no data tag left (what data is it sending?).

2.4. Solution

Solution: use CAN 2.0B extended frames, which give a 29-bit ID instead of 11 bits. Higher-level standards such as SAE J1939 are built on top of those extended frames and split the ID in a similar way. Now we have more room to work with.

Rather than the split above, we can split it like this:

- 3 bits for priority
- 8 bits for Command (data type)
- 8 bits for Receiver ID
- 8 bits for Sender ID

Now we have 256 peripherals that can talk to each other, with 256 commands to share the data with everyone.

There are even 2 spare bits left, to spend on something in the future.

3. How do you define the structure of the message?

A CAN frame already has 8 bytes of data that you can use, and this is how I will use them.

- Byte 0 The first byte is the sequence counter (0-255). It goes up by one on every message of that type, so the receiver can see when a frame was dropped. I don't need a length field or a checksum here, because CAN already sends the data length (DLC) and a CRC in hardware and rejects corrupted frames by itself.
- Byte 1 - 7 The remaining bytes are for the data itself. It can be sensor readings, motor commands, or alerts.

3.1. How to pack different data (just examples)

Because different sensors send different types of numbers, we define strict rules for how those 56 data bits are sliced up based on the Command ID (which we defined in the 29-bit CAN ID header). So we didn't just fill the whole 56 bits of data with random values.

3.1.1. For example, Motor Drive Command (Command ID: 0x12)

The Central Computer needs to send speed commands to the left and right wheel motors. We can pack two 16-bit signed integers (shorts) and an 8-bit mode flag into the payload:

- Byte 0: Sequence Counter (0-255) to detect dropped packets
- Bytes 1-2: Left Motor Target Speed (16-bit signed integer)
- Bytes 3-4: Right Motor Target Speed (16-bit signed integer)
- Byte 5: Control Mode (8-bit enum: 0x00 = Idle, 0x01 = Speed Control, 0x02 = Torque Control)
- Bytes 6-7: Unused / Reserved for future growth

3.1.2. Another example, receiving IMU Sensor Readings (Command ID: 0x35)

The IMU needs to send high-precision acceleration data (X, Y, Z axes)

- Byte 0: Sequence Counter, just like before
- Bytes 1-2: X-axis acceleration (16-bit)
- Bytes 3-4: Y-axis acceleration (16-bit)
- Bytes 5-6: Z-axis acceleration (16-bit)
- Byte 7: Unused

4. How do you handle receiving and processing packets at high speed?

For this step, I will use a few tricks so the boards can keep up with the fast data. The main idea is to do as little work as possible at the moment a frame arrives, and do the heavy work later when there is time.

I will split the receiving process into a few steps. Each step removes some work from the next one.

4.1. Step 1: Let the hardware filter first

Every CAN controller has acceptance filters (filters and masks). These are set up once at startup and tell the controller which IDs to keep and which to throw away.

Because of the ID layout I defined earlier, each board can set its filter to only accept the id it needs.

Everything else gets dropped by the hardware itself before the CPU even sees it.

4.2. Step 2: Keep the interrupt as short as possible

I will make the interrupt do only one thing: copy the frame into a software buffer and leave. It does not do any of the following:

- Parsing the data.
- Doing calculations (especially floating point).
- Printing debug messages.
- Waiting for anything.

It works like a mail room. The receptionist does not open and read every letter when it arrives. They just drop it in the tray, so they are ready for the next delivery. Reading the letters happens later.

4.3. Step 3: Ring buffer (circular buffer)

The software buffer that the interrupt copies into is a ring buffer. It is a fixed-size array with two pointers:

- The head, where the interrupt writes new frames.
- The tail, where the main loop reads frames out.

When a pointer reaches the end of the array, it wraps back to the start, like a circle.

Why a ring buffer:

- No dynamic memory: the size is fixed at compile time, so it can never run out of memory at runtime.

- No locking needed: only the interrupt writes and only the main loop reads, so they never fight over the same data.
- Fast : adding or removing a frame is just copying 8 bytes and moving a pointer.

4.4. Step 4: Dispatch table instead of a long if/else chain

When the main loop takes a frame out of the ring buffer, it has to decide what to do with it. The slow way is a long chain of “if command is this, else if command is that...”. The more message types we add, the slower it gets.

Instead, I use a dispatch table: an array of 256 entries, one for each possible Command ID. Each entry points to the function that handles that message. The Command ID is used directly as the index, so finding the right handler takes the same amount of time whether we have 5 message types or 200.

For example

- Frame with Command ID `0x12` comes in → go to entry `0x12` → run the Motor Drive Command handler.
- Frame with Command ID `0x35` comes in → go to entry `0x35` → run the IMU handler.
- Frame with an unknown Command ID → the entry is empty → count it as an error and ignore it.

This also helps with changing requirements. Adding a new message type means adding one entry in the table, without touching the rest of the code.

4.5. Step 5: Handle priority in software too

The CAN bus already makes sure high-priority frames win arbitration on the wire. But if all frames end up in the same ring buffer, an Emergency Stop could still be stuck behind 50 IMU frames waiting to be processed.

So I use two ring buffers:

- High priority (priority 0-1): emergency stop, alerts, drive commands.
- Normal priority (priority 2-7): sensor data, heartbeats, logging.

The main loop always empties the high-priority buffer first. For the emergency stop specifically, the motor can be disabled directly inside the interrupt, because stopping safely is more important than keeping the interrupt short.

4.6. Step 6: Latest value vs every value

Not every message needs to be kept.

- **Only the latest value matters:** motor encoder position, battery voltage, heartbeats. If two frames arrive before we process them, we only care about the newest one, so it simply overwrites the old value. The encoder is safe here because it sends an absolute count, so skipping one reading does not make the next one less correct.
- **Every value matters:** commands, alerts, and IMU readings. A missed command or alert can be dangerous. The IMU belongs here too, because the software adds up every sample over time to estimate the position, so a dropped sample directly makes the position less accurate.

Overwriting values that go stale anyway saves processing time and keeps the buffers from filling up with outdated data.

4.7. Step 7: Check every frame on receive

Before the data is used, each handler does some quick checks:

- Length check: the data length must match what that Command ID is supposed to have. If not, the frame is rejected and counted as an error.
- Sequence counter check: if the counter jumps (for example from 41 to 44), we know 2 frames were lost, and we count them.
- Timeout check: every important message has a maximum age. If the drive board does not get a new drive command within, for example, 100 ms, it stops the motors by itself. If a board's heartbeat stops, the central computer marks that board as offline.

The timeout check is a safety feature. If the central computer crashes or a cable comes loose, the rover stops on its own instead of driving away with the last command it received.

4.8. Summary of the full receive path

1. The frame arrives on the bus.
2. The hardware filter drops frames not meant for this board.
3. The frame lands in the hardware FIFO and the interrupt fires.
4. The interrupt copies it into the high or normal priority ring buffer and returns.
5. The main loop takes frames out, high priority first.
6. The dispatch table sends each frame to the correct handler.
7. The handler checks the length, sequence counter and timeouts, then uses the data.

5. How do you handle multiple types of packets to send or receive?

On the rover, lots of different messages are sent at the same time. Motor data, sensor data, alerts, commands, and more. They are not all the same, so I don't treat them all the same way.

5.1. Three kinds of messages

I put every message into one of three groups.

- Regular messages: sent again and again, like a clock ticking. For example the motor encoder or the IMU. Even if nothing changed, they still get sent, so the other boards always have fresh data.
- Alert messages: only sent when something happens. For example an emergency stop, or a motor getting too hot.
- Question and answer messages: one board asks something, and another board answers. For example the central computer asking the arm board "where is your arm right now?"

It is a bit like a newsroom. Some news is sent every hour (the weather), some news is sent the moment it happens (breaking news), and some is only sent when someone calls and asks for it.

5.2. One list of all messages

I keep one list with every message the rover uses. All boards use the same list, so everyone agrees on what each message means.

For each message, the list says what its Command ID is, how important it is, who sends it, and how often.

5.2.1. For example

Message	Command ID	Priority	Who sends it	How often
Emergency stop	0x01	0	Any board	When needed
Motor drive command	0x12	1	Central computer	50 times per second
Motor encoder	0x20	2	Drive board	100 times per second
IMU acceleration	0x35	2	Sensor board	200 times per second
Battery voltage	0x40	5	Sensor board	1 time per second
Heartbeat	0x7F	6	Every board	10 times per second

If we need a new message later, we just add a new line to this list.

5.3. How regular messages are sent

Each board has a small timer, like an alarm clock. Every message has its own alarm. When the alarm goes off, the message is sent, and the alarm is set again.

The alarms don't all go off at the exact same moment. If they did, all messages would try to use the bus at once and some would have to wait. So each message starts a little bit later than the other ones. This spreads the messages out, like cars leaving a parking lot one by one instead of all at once.

5.4. How alert messages are sent

Alerts don't wait for a timer. They are sent right away. They also have the highest priority, so they always go first on the bus.

For important alerts, the board sends the message more than once. That way, if one message gets lost, the others still arrive.

5.5. How question and answer messages are sent

A board sends a question to one specific board, using that board's ID. The other board sends the answer back to whoever asked.

If no answer comes back after a short time, the question is asked again. If there is still no answer after a few tries, we know that board has a problem.

5.6. Waiting in line to be sent

A board can only send a few messages at the same time. The rest have to wait in a line.

This line is not "first come, first served". The most important message always goes to the front. Just like in a hospital, where the most urgent patient is helped first, not the one who came in first. So an emergency stop never has to wait behind normal sensor messages.

5.7. Receiving different messages

This is already explained in the previous section. In short: each board only listens to the messages it needs, and every type of message goes to its own piece of code that knows how to handle it.

5.8. Don't overload the bus

The bus is like a road. Every message is a car on that road. If there are too many cars, there will be a traffic jam, and the less important messages will get stuck.

So before sending more messages, or sending them more often, I first calculate if the bus can handle it. A good rule is to keep the bus less than about 50–70% full, so there is always some room left.

6. How do you handle a change in requirements?

Requirements always change during a project. A new sensor gets added, someone needs data faster, or a board gets replaced. So I don't try to design a system that never changes. I design a system where changes are easy and safe.

It is like building a house. If you already put some extra power sockets and empty pipes in the walls, adding a new room later is easy. If you didn't, you have to break walls open.

6.1. Leave empty space from the start

I already left some free room in the design:

- The CAN ID has 2 spare bits that are not used yet.
- There are 256 possible Command IDs, and the rover only uses a few of them.
- Some messages have unused bytes at the end, like bytes 6–7 in the Motor Drive Command.

So when something new is needed, there is already space for it. We don't have to change the whole design.

6.2. Change the message list first

Every change starts in the message list from the previous section. Because all boards are built from that same list, a change there reaches every board. Nobody has to remember to update 5 different places by hand.

6.2.1. For example, adding a temperature sensor

The team wants to measure the motor temperature.

1. Pick a free Command ID, for example 0x50.
2. Add a new line to the message list: who sends it, how often, and what the bytes mean.
3. The sensor board starts sending it, and the boards that need it add a handler for it.

The other boards don't need to do anything. They simply ignore the new message, because their filters don't let it through.

6.3. Add new things, don't change old things

If a message needs to change, I don't change the meaning of the old one. I make a new message next to it instead.

If I change what the bytes of an old message mean, a board that is not updated yet will read the new data the old way. That gives wrong numbers without any error. Adding a new message keeps the old boards working until they are updated too.

For small changes, the unused bytes can be used. Old boards just ignore those bytes, so nothing breaks.

6.4. Every board says its version

Every board sends its firmware version in its heartbeat message. The central computer checks this when the rover starts. If one board has an old version that doesn't match the rest, we see a warning right away. This way we don't spend hours debugging a problem that was just an old version on one board.

6.5. Check the bus before making things faster

When someone asks for more data, like faster encoder updates, I first check if the bus can handle it. The message list shows how often every message is sent, so I can calculate the new bus load before changing anything.

If it doesn't fit, there are a few options, from easiest to hardest:

- Send less important messages less often. Battery voltage only needs to be sent once per second, not 10 times.
- Only send a message when the value actually changes, instead of all the time.
- Pack data smarter, for example use smaller numbers when full precision is not needed.
- Make the bus faster, or switch to CAN FD, which is faster and fits up to 64 bytes per message.
- Split the rover into two buses, for example one for the motors and one for the sensors.

6.6. Test the change before putting it on the rover

Before a change goes on the real rover, I test it:

- Record the CAN traffic from a normal drive and play it back to the changed board. This checks that the board still handles all the old messages correctly.
- Watch the bus load and the lost-message counters after the change. They show right away if the change made things worse.

6.7. Make updating easy

If every board has to be opened up and plugged into a laptop to update it, people will skip updates, and boards end up with different versions. So each board gets a bootloader that can receive new firmware over the CAN bus itself. Then all boards can be updated from the central computer in one go.

7. Are there other things to keep in mind?

A good message design is only part of the story. On a real rover, there are also motors, vibrations and things that break.

7.1. Wiring

Many CAN problems are actually wiring problems.

- The bus needs a 120 ohm resistor at both ends, or the signal echoes and gets corrupted.
- The two CAN wires should be twisted together to block noise.
- The faster the bus, the shorter the cable can be (about 40 meters at 1 Mbit/s).
- All boards should share the same ground.

7.2. Motor noise

Motors create electrical noise that can corrupt messages. Keep CAN cables away from motor power cables, and use shielded cable in noisy places.

7.3. Vibration

A rover shakes while driving. A slightly loose connector can lose contact for a moment, which looks like random lost messages. Use locking connectors and secure the cables.

7.4. Broken boards

If a board makes too many errors, CAN switches it off the bus by itself (“bus-off”), so it can’t ruin the bus for others. The software should notice this and try to reconnect. Each board should also have a limit on how many messages it sends, so a buggy board can’t flood the bus.

7.5. Safety

If communication fails, the rover must stop, not keep driving. The drive board stops the motors if no new command arrives in time. There should also be a physical emergency stop that cuts motor power directly, because a CAN emergency stop can’t be delivered if the bus itself is broken.

7.6. Timestamps

To combine encoder and IMU data correctly, the central computer needs to know when each value was measured, not just when it arrived. Sensor boards can add a timestamp, using a clock that the central computer keeps in sync.

7.7. Bus health

Each board counts sent messages, lost messages, full buffers and CAN errors, and reports them in its heartbeat. This way, problems are spotted early.

7.8. Documentation

The message list and wiring diagram should be kept up to date, so anyone can quickly understand the system.

8. Scenario: the communication system is too slow

A month before the deadline, the bus is too slow and many messages are lost. Control wants faster encoder updates, and software wants more IMU messages.

I would measure first, I won’t blindly guess. With only a month left, there is no time to try random fixes. First find out where the messages are lost, then fix that exact spot.

8.1. Part 1: Start the debugging

8.1.1. Step 1: Look at the bus

Connect a CAN analyzer to the bus and let the rover run normally. Check two things:

- Bus load: how full is the bus? If it is close to 100%, the bus is simply overloaded.
- Error frames: are there many errors on the bus? If yes, messages are being corrupted, which points to a wiring or noise problem.

8.1.2. Step 2: Do the math

Take the message list and calculate how much traffic all messages should create together. If the numbers already add up to more than the bus can carry, the problem is not a bug, the design simply asks for too much.

With the numbers from the message list in section 5, it works out like this. An extended CAN frame with 8 data bytes is about 150 bits on the wire, including the ID, the CRC and the stuffing bits. At 1 Mbit/s that means the bus can carry roughly $1,000,000 / 150 \approx 7,000$ frames per second.

The message list adds up to about 400 frames per second:

Message	Frames per second
Motor drive command	50
Motor encoder	100
IMU acceleration	200
Battery voltage	1
Heartbeat (5 boards $\times$ 10)	50
Total	≈ 401

That is about $401 / 7,000 \approx 6\%$ bus load at 1 Mbit/s. The same traffic at 125 kbit/s would be about 50%.

This is worth writing down, because it already tells me a lot. With these rates the bus should not be anywhere near full at 1 Mbit/s. So if messages are still being lost, the cause is most likely on the receiving side (a board that is too slow and overflows its buffer), in the wiring, or the bus is actually running slower than we think it is.

8.1.3. Step 3: Find where the messages are lost

A message can get lost in three places:

- When sending: the board's send queue is full, so messages are dropped before they reach the bus. Normal-priority messages might also never get a turn, because higher-priority messages keep winning.
- On the wire: the message gets corrupted by noise or bad wiring.
- When receiving: the board is too slow to read its messages, so its buffer overflows.

The counters in every heartbeat (sent, lost, full buffers, errors) show which of these is happening, and on which board.

8.1.4. Step 4: Check the wiring

If there are many errors on the wire:

- Turn the power off and measure the resistance between CAN High and CAN Low. It should be about 60 ohm (two 120 ohm resistors in parallel). A different value means termination is missing or wrong.
- Look at the signal with an oscilloscope. It should have clean, square edges.
- Compare with the motors on and off. If the errors only happen when the motors run, it is noise from the motors.
- Check that all boards use exactly the same bus speed settings.

8.1.5. Step 5: Test one board at a time

If it is still unclear, take the boards to the bench and connect them one by one. When the problem appears after adding a certain board, that board is the cause.

8.2. Part 2: If the problem persists

If the wiring is fine and the bus is simply too full, the system has to send less, or get more room. I would try these options from easiest to hardest, because time is short.

8.2.1. Send less

- Send less important messages less often. Battery voltage only needs to be sent once per second, not 10 times.
- Only send some messages when the value changes.
- Pack the data smarter, for example several encoder values in one message.

8.2.2. Do the work closer to the data

The control team wants fast encoder updates because the control loop runs on the central computer. If the fast motor control loop runs on the drive board itself, the encoder data never has to go over the bus. The central computer only sends the target speed.

The same idea works for the IMU. It can be connected directly to the central computer (for example with SPI or USB), so its many messages don't use the shared bus at all.

8.2.3. Get more room

- Raise the bus speed, if the cables are short enough.
- Switch to CAN FD, which is faster and fits up to 64 bytes per message.
- Split the rover into two buses, for example one for the motors and one for the sensors.

These options need new hardware or firmware on all boards, so with one month left they are the last choice.

8.2.4. Talk with the teams

Not every wish can be met, so I would sit down with control and software and ask what rates they really need, not what would be nice to have. Then we agree on the numbers, update the message list, and check the bus load again before building it.